

Ejercicio Individual de la sesión

Se configura un proyecto en blanco con React-Native expo:

```
npx create-expo-app@latest PlantillaS9 --template blank
```

Se inicia con el siguiente código en App.js, donde se usa un patrón de diseño para envolver con diversos componentes nuestra aplicación móvil, que empezaría en MainContent

```
import { View, Text, StyleSheet, Image } from 'react-native';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
import { PaperProvider } from 'react-native-paper';
import MainContent from './src/components/MainContent';
import { SQLiteProvider } from 'expo-sqlite';

import { initDB } from './src/database/initDB';
import { GestureHandlerRootView } from 'react-native-gesture-handler';
export default function App() {
  return (

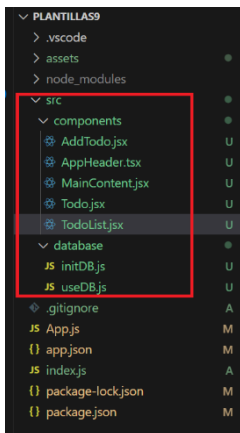
    <SQLiteProvider databaseName='tasks.db' onInit={initDB}>
      <PaperProvider>
        <GestureHandlerRootView>
          <MainContent />
          <StatusBar style="auto" />
        </GestureHandlerRootView>
      </PaperProvider>
    </SQLiteProvider>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Se instalan las dependencias necesarias

```
npm install react-native-paper
npx expo install react-native-safe-area-context
npx expo install expo-sqlite
npx expo install react-native-gesture-handler
npx expo install expo-linear-gradient
npx expo install react-native-svg
```

Crear una carpeta src para contener todo el código, con la estructura indicada en la figura.



Dentro del archivo `initDB.js`, se coloca el siguiente código para crear la estructura de la base de datos (básicamente una tabla):

```
export async function initDB(db) {
  await db.execAsync(`
    CREATE TABLE IF NOT EXISTS tasks (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      description TEXT NOT NULL,
      createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
      isCompleted BOOLEAN DEFAULT false
    );
  `);
}
```

A continuación, se coloca el siguiente código dentro del archivo `useDB.js`, el cual contiene todas las funciones para ejecutar las operaciones CRUD en una base de datos

```
import { useSQLiteContext } from "expo-sqlite";
export function useDB() {

  const database = useSQLiteContext();

  async function findAll() {
    try {
      const query = "SELECT * FROM tasks";
      const result = await database.getAllAsync(query);
      return result;
    } catch (error) {
      console.error(error);
    }
  }

  //search records by text into description
  async function searchByDescription(text) {
    try {
      const query = "SELECT * FROM tasks WHERE description LIKE ?";
      const result = await database.getAllAsync(query, `_${text}%`);
      return result;
    } catch (error) {
      console.error(error);
    }
  }

  //create a record
  async function createOne(task) {
    const statement = await database.prepareAsync(
      "INSERT INTO tasks (description) VALUES ($description)"
    );
    try {
      const result = await statement.executeAsync({
        $description: task.description,
      });
      // const insertedRowId = result.lastInsertRowId.toLocaleString();
    } catch (error) {
      console.error(error);
    }
  }
}
```

```

        // return { insertedRowId };
        return result;
    } catch (error) {
        console.error(error);
    } finally {
        await statement.finalizeAsync(); //best practice, to finalize prepared statements
    }
}

//update record toggling isCompleted state
async function toggleIsCompleted(id, newState) {
    const query = "UPDATE tasks SET isCompleted = ? WHERE id = ?";
    const result = await database.getFirstAsync(query, !newState, id);
}

//delete a record
async function deleteOne(id) {
    try {
        const result = await database.execAsync(
            `DELETE FROM tasks WHERE id = ${id}`
        );
        // const query = "DELETE FROM tasks WHERE id = ?";
        // const result = await database.getFirstAsync(query, id);
    } catch (error) {
        console.error(error);
    }
}

return {
    findAll,
    searchByDescription,
    createOne,
    toggleIsCompleted,
    deleteOne
};
}

```

A continuación, se escribe el siguiente código dentro del archivo MainContent.jsx, el cual es un componente funcional que retorna JSX. Se retorna un SafeAreaView con estilo {flex: 1} para que ocupe toda la pantalla disponible. Dentro del SafeAreaView se renderiza el componente TodoList, que será mostrado respetando las áreas seguras del dispositivo:

```

import { SafeAreaView } from 'react-native-safe-area-context';
import TodoList from './TodoList';

export default function MainContent() {
    return (
        <SafeAreaView style={{flex:1}}>
            <TodoList />
        </SafeAreaView>
    );
}

```

A continuación, se coloca el siguiente código que define un componente funcional llamado TodoList que administra una lista de tareas (todos) con operaciones para agregar, eliminar, marcar como completadas y filtrar basándose en una búsqueda :

```

import React, { useState, useEffect } from 'react';
import { View, FlatList, StyleSheet } from 'react-native';
import Todo from './Todo';
import { LinearGradient } from 'expo-linear-gradient';
import AppHeader from './AppHeader';
import AddTodo from './AddTodo';
import { useDB } from '../database/useDB';

const TodoList = () => {

```

```

const db = useDB();
const [todos, setTodos] = useState([]);
const [text, setText] = useState("");
const [query, setQuery] = useState("");

const addTodo = async () => {
  if (text.trim()) {
    await db.createOne({ description: text });
    setText("");
    await list();
  }
};

const deleteTodo = async (id) => {
  try {
    await db.deleteOne(id);
    list();
  } catch (error) {
    console.error(error);
  }
};

const toggleIsCompleted = async (id, state) => {
  try {
    await db.toggleIsCompleted(id, state);
    list();
  } catch (error) {
    console.error(error);
  }
};

const list = async () => {
  try {
    const results = await db.findAll();
    setTodos(results);
  } catch (error) {
    console.error(error);
  }
};

const filteredTodos = todos.filter(todo => todo.description.toLowerCase().includes(query.toLowerCase()));

useEffect(() => {
  list();
}, []);

return (
  <LinearGradient
    colors={['#dadde4', '#c0d3fd', '#76a9d5', '#512da8']}
    style={styles.fullWidth}>
    <AppHeader />
    <View style={{ flex: 1, padding: 16 }}>
      <AddTodo text={text} query={query} setText={setText} addTodo={addTodo} setQuery={setQuery} />

      <FlatList
        data={filteredTodos}
        keyExtractor={(item) => item.id.toString()}
        showsVerticalScrollIndicator={false}
        renderItem={({ item }) =>
          <Todo {...item} deleteTodo={deleteTodo} toggleIsCompleted={toggleIsCompleted}>{item.description}</Todo>
        }
      />

    </View>
  </LinearGradient >
);

const styles = StyleSheet.create({
  fullWidth: {

```

```

      flex: 1,
    },
  });

export default TodoList;

```

A continuación, se coloca el siguiente código en el archivo AppHeader.jsx, el cual corresponde al componente AppHeader. Este crea una cabecera gráfica para una pantalla, con un fondo morado curvado hecho con SVG que ocupa todo el ancho visible seguro del dispositivo, y sobre ese fondo dibuja un texto "tareApps" en color claro, posicionado encima en la esquina izquierda.

```

import { StyleSheet, View } from 'react-native'
import { Text } from 'react-native-paper'
import { Svg, Path } from 'react-native-svg'
import { useSafeAreaFrame } from 'react-native-safe-area-context'
import React from 'react'

export default function AppHeader() {
  const frame = useSafeAreaFrame()
  return (
    <View>
      <Svg width={frame.width} height={'120'} viewBox={'0 0 ${frame.width} 100'} preserveAspectRatio="none" fill={'#512da8'}>
        <Path
d="M0,0V46.29c47.79,22.2,103.59,32.17,158,28,70.36-5.37,136.33-33.31,206.8-37.5C438.64,32.43,512.34,53.67,583,72.05c69.27,18,138.3,24.88,209.4,13.08,36.15-6,69.85-17.84,104.45-29.34C989.49,25,1113-14.29,1200,52.47V0Z" opacity={80} />
        </Svg>
        <Text variant='displaySmall' style={styles.headerTitle}>tareApps</Text>
      </View>
    )
  }

const styles = StyleSheet.create({
  headerTitle: {
    color: "#f2e9e9",
    position: 'absolute',
    left: 20
  }
})

```

A continuación, se adiciona el siguiente código en el archivo AddTodo.jsx. Este componente llamado AddTodo permite agregar o buscar tareas con un campo de texto y dos botones que cambian según si estás buscando (query) o no (agrega tareas).

```

import { Keyboard, StyleSheet, View } from 'react-native';
import { TextInput, Button } from 'react-native-paper';

const AddTodo = ({ text, query, setText, addTodo, setQuery }) => {
  function viewAll() {
    setQuery("");
    setText("");
    Keyboard.dismiss();
  }
  function handleSearch() {
    setQuery(text);
    Keyboard.dismiss();
  }
  return (
    <View style={styles.container}>
      <TextInput
        multiline
        mode='flat'
        activeOutlineColor='tomato'
        textColor='black'
        value={text}
        onChangeText={setText}

```

```

        label="Agregar o Buscar tareas"
        right={<TextInput.Icon icon="magnify" onPress={handleSearch} />}
      />
    {!query ?
      (<Button mode='contained' collapsable onPress={addTodo} >Agregar tarea</Button>)
      :
      (<Button mode='contained' onPress={viewAll} >Ver Todo</Button>)
    }
  </View>
);
};

export default AddTodo;

const styles = StyleSheet.create({
  container: {
    rowGap: 10,
    justifyContent: "space-between",
    marginBottom: 10
  }
});

```

A continuación, se agrega el siguiente código al archivo `Todo.js`. Este código es un componente funcional React Native llamado `Todo` que representa una tarea con funcionalidad para completar o eliminar mediante gestos de deslizamiento (swipe) y botones :

```

import { StyleSheet, View, TouchableOpacity, Alert, Animated } from 'react-native';
import { Text } from 'react-native-paper';
import MaterialCommunityIcons from '@expo/vector-icons/MaterialCommunityIcons';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
const SWIPE_THRESHOLD = 100;
const Todo = ({ id, description, isCompleted, createdAt, deleteTodo, toggleIsCompleted }) => {
  const showDetails = () => { };
  const handleDelete = (id) => {
    Alert.alert(
      'Eliminar tarea',
      `¿Estás seguro de que deseas eliminar ${id}?`,
      [
        {
          text: 'Cancelar',
          style: 'cancel',
        },
        {
          text: 'Eliminar',
          style: 'destructive',
          onPress: () => deleteTodo(id)
        }
      ]
    );
  };

  //lógica de acciones por desplazamiento
  const translateX = new Animated.Value(0);

  const panGesture = Gesture.Pan()
    .onUpdate((event) => {
      translateX.setValue(event.translationX);
    })
    .onEnd((event) => {
      const { translationX } = event;
      if (translationX < -SWIPE_THRESHOLD * 2) {
        deleteTodo(id);
      } else if (translationX > SWIPE_THRESHOLD * 2) {
        toggleIsCompleted(id, isCompleted);
      }
      Animated.spring(translateX, {
        toValue: 0, // Regresa a la posición original.
        bounciness: 10, // Grado de rebote al regresar.
      });
    });

```

```

        useNativeDriver: false
    }).start();
  });
  // `redLayerOpacity` se utiliza para controlar la visibilidad de la capa roja que indica eliminación.
  // A medida que el usuario realiza el swipe hacia la izquierda, la capa se vuelve más visible.
  const redLayerOpacity = translateX.interpolate({
    inputRange: [-SWIPE_THRESHOLD * 2, 0], // Rango de movimiento para la interpolación.
    outputRange: [1, 0], // La opacidad cambia de 1 (completamente visible) a 0 (invisible).
    extrapolate: 'clamp', // Limita los valores interpolados al rango definido.
  });

  // Similar a `redLayerOpacity`, `deleteIconOpacity` controla la visibilidad del ícono de eliminación.
  const deleteIconOpacity = translateX.interpolate({
    inputRange: [-SWIPE_THRESHOLD * 2, 0],
    outputRange: [1, 0],
    extrapolate: 'clamp',
  });

  // `purpleLayerOpacity` se utiliza para la capa púrpura, que indica que el todo será completado si el swipe es suficientemente
  largo hacia la derecha.
  const purpleLayerOpacity = translateX.interpolate({
    inputRange: [0, SWIPE_THRESHOLD * 2],
    outputRange: [0, 1],
    extrapolate: 'clamp',
  });

  // `checkIconOpacity` controla la visibilidad del ícono que marca el todo como completado.
  const checkIconOpacity = translateX.interpolate({
    inputRange: [0, SWIPE_THRESHOLD * 2],
    outputRange: [0, 1],
    extrapolate: 'clamp',
  });

  // `animatedStyle` se utiliza para aplicar transformaciones animadas al todo.
  // Aquí se aplica una transformación de traslación horizontal basada en `translateX`.
  const animatedStyle = {
    transform: [{ translateX }]
  };

  return (
    <View style={styles.container}>
      <Animated.View style={[styles.redLayer, { height: 40, opacity: redLayerOpacity }]}>
        <Animated.View style={[styles.deleteIconContainer, { opacity: deleteIconOpacity }]}>
          <MaterialCommunityIcons name="trash-can-outline" size={24} color="whitesmoke" />
        </Animated.View>
      </Animated.View>

      <Animated.View style={[styles.purpleLayer, { opacity: purpleLayerOpacity, height: 40 }]}>
        <Animated.View style={[styles.checkIconContainer, { opacity: checkIconOpacity }]}>
          <MaterialCommunityIcons name="check-circle-outline" size={24} color="whitesmoke" />
        </Animated.View>
      </Animated.View>

      <GestureDetector gesture={panGesture}>
        <Animated.View style={[styles.todoContainer, animatedStyle]}>
          <TouchableOpacity
            onPress={showDetails}
            onLongPress={() => toggleIsCompleted(id, isCompleted)}
          >

            <Text style={[isCompleted && styles.completed, { width: 300 }]} variant='bodyLarge'>
              {description.length > 35 ? description.slice(0, 35) + '...' : description}
            </Text>
          </TouchableOpacity>

          <TouchableOpacity>
            <MaterialCommunityIcons name="trash-can-outline" size={24} color="black"
              onPress={() => handleDelete(id)} />
          </TouchableOpacity>
        </Animated.View>
      </GestureDetector>
    </View>
  );

```

```

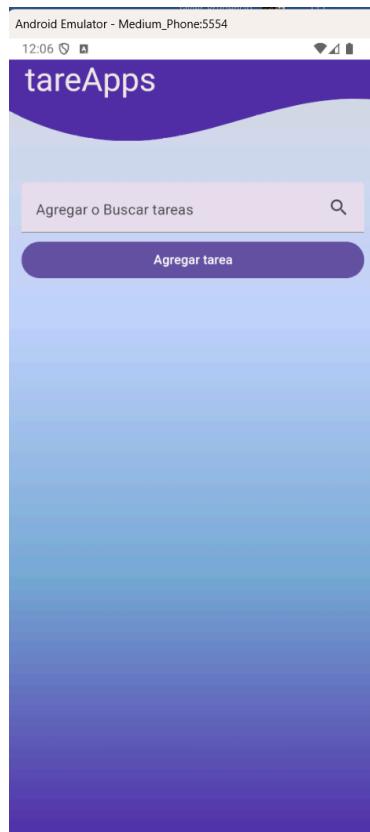
    </View >
  );
};

export default Todo;

const styles = StyleSheet.create({
  container: {
    position: 'relative', // Posición relativa para permitir que las capas de color se superpongan correctamente al contenedor principal del todo.
  },
  todoContainer: {
    flexDirection: 'row', // Alinea el texto y los íconos en una fila horizontal.
    justifyContent: 'space-between', // Distribuye el espacio entre el texto y el ícono de eliminación.
    alignItems: 'center', // Alinea verticalmente el contenido en el centro.
    paddingVertical: 8, // Espaciado vertical interno del todo.
    marginTop: 20, // Margen superior entre cada todo.
    backgroundColor: 'rgb(237, 221, 245)', // Color de fondo del todo.
    borderRadius: 12, // Bordes redondeados para un diseño más amigable.
    paddingLeft: 20, // Espaciado a la izquierda del texto.
    paddingRight: 20, // Espaciado a la derecha del texto.
  },
  completed: {
    textDecorationLine: 'line-through', // Aplica una línea a través del texto para indicar que el todo está completado.
    color: '#512da8', // Cambia el color del texto para indicar su estado de completado.
  },
  redLayer: {
    position: 'absolute', // Posición absoluta para que la capa roja cubra toda el área del todo cuando se realiza el swipe.
    top: 20, // Alineación superior basada en el margen superior del todo.
    right: 0, // La capa roja se alinea al borde derecho del todo.
    left: 0, // La capa roja se extiende hasta el borde izquierdo del todo.
    backgroundColor: 'red', // Color de fondo rojo para indicar una acción destructiva (eliminar).
    borderRadius: 12, // Bordes redondeados para que coincidan con los del todo.
  },
  deleteIconContainer: {
    position: 'absolute', // Posiciona el ícono de eliminación dentro de la capa roja.
    right: 15, // Desplazado ligeramente hacia la izquierda desde el borde derecho.
    top: '20%', // Ajustado verticalmente en la capa.
  },
  purpleLayer: {
    position: 'absolute', // Posición absoluta similar a la capa roja.
    top: 20,
    right: 0,
    left: 0,
    backgroundColor: 'blueviolet', // Color de fondo púrpura para indicar una acción constructiva (completar).
    borderRadius: 12,
  },
  checkIconContainer: {
    position: 'absolute', // Posiciona el ícono de check dentro de la capa púrpura.
    left: 15, // Desplazado ligeramente hacia la derecha desde el borde izquierdo.
    top: '20%', // Ajustado verticalmente en la capa.
  }
});

```

La aplicación debe tener la siguiente apariencia



Anexo

```
1  export async function initDB(db) {  
2    await db.execAsync(`  
3      CREATE TABLE IF NOT EXISTS tasks (  
4        id INTEGER PRIMARY KEY AUTOINCREMENT,  
5        description TEXT NOT NULL,  
6        createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7        isCompleted BOOLEAN DEFAULT false  
8      );  
9    `);  
10 }
```

Vamos a analizar detalladamente el código React Native que crea una base de datos con SQLite mediante la función `initDB(db)`. Esta función usa una operación asíncrona para crear una tabla llamada `tasks` si no existe, con las columnas `id`, `description`, `createdAt` y `isCompleted`.

Explicación paso a paso:

1. `export async function initDB(db) { ... }`
 - Define una función asíncrona llamada `initDB` que recibe como parámetro la conexión a la base de datos (`db`).

- La palabra clave `async` indica que dentro de esta función usaremos operaciones asíncronas con `await`, para esperar resultados sin bloquear la ejecución.
2. `await db.execAsync(CREATE TABLE IF NOT EXISTS tasks (...));`
- Aquí se ejecuta una sentencia SQL para crear la tabla `tasks` si no existe.
 - `execAsync` es una función que ejecuta comandos SQL de manera asíncrona usando promesas.
 - La sentencia SQL incluye:
 - `id INTEGER PRIMARY KEY AUTOINCREMENT`: columna `id` clave primaria que se incrementa automáticamente para cada fila.
 - `description TEXT NOT NULL`: columna para la descripción de la tarea, texto obligatorio.
 - `createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP`: columna que guarda la fecha y hora de creación automática.
 - `isCompleted BOOLEAN DEFAULT false`: columna booleana que representa si la tarea está completada, inicia en `false`.
3. El uso de `CREATE TABLE IF NOT EXISTS` permite que esta función se ejecute sin errores repetidos si la tabla ya fue creada anteriormente, útil para inicializar la base de datos la primera vez que se abre la app.

¿Por qué usar una función asíncrona aquí?

Dado que la creación de tablas implica operaciones de disco que pueden tardar, `async/await` facilita manejar el código sin bloquear la interfaz y permite encadenar operaciones posteriores que dependan de la creación exitosa de la tabla.

Esta función es ideal para colocarla en la fase inicial de la aplicación (por ejemplo, en el `App.js` o contexto de la base de datos) para asegurarnos que la tabla esté lista antes de manipular datos.

```

1  import { useSQLiteContext } from "expo-sqlite";
2  export function useDB() {
3
4      const database = useSQLiteContext();
5
6      async function findAll() {
7          try {
8              const query = "SELECT * FROM tasks";
9              const result = await database.getAllAsync(query);
10             return result;
11         } catch (error) {
12             console.error(error);
13         }
14     }
15
16     //search records by text into description
17     async function searchByDescription(text) {
18         try {
19             const query = "SELECT * FROM tasks WHERE description LIKE ?";
20             const result = await database.getAllAsync(query, `${text}%`);
21             return result;
22         } catch (error) {
23             console.error(error);
24         }
25     }
26
27     //create a record
28     async function createOne(task) {
29         const statement = await database.prepareAsync(
30             "INSERT INTO tasks (description) VALUES ($description)"
31         );
32         try {
33             const result = await statement.executeAsync({
34                 $description: task.description,
35             });
36             // const insertedRowId = result.lastInsertRowId.toLocaleString();
37             // return { insertedRowId };
38             return result;
39         } catch (error) {
40             console.error(error);
41         } finally {
42             await statement.finalizeAsync(); //best practice, to finalize prepared statements
43         }
44     }
45
46     //update record toggling isCompleted state
47     async function toggleIsCompleted(id, newState) {
48         const query = "UPDATE tasks SET isCompleted = ? WHERE id = ?";
49         const result = await database.getFirstAsync(query, !newState, id);
50     }
51
52     //delete a record
53     async function deleteOne(id) {
54         try {
55             const result = await database.execAsync(
56                 `DELETE FROM tasks WHERE id = ${id}`
57             );
58             // const query = "DELETE FROM tasks WHERE id = ?";
59             // const result = await database.getFirstAsync(query, id);
60         } catch (error) {
61             console.error(error);
62         }
63     }
64
65     return {
66         findAll,
67         searchByDescription,
68         createOne,
69         toggleIsCompleted,
70         deleteOne
71     };
72
73 }

```

El código mostrado es un hook personalizado en React Native que utiliza la librería `expo-sqlite` para interactuar con una base de datos SQLite de manera asíncrona. Este hook llamado `useDB` provee funciones para realizar operaciones

CRUD (Crear, Leer, Actualizar, Borrar) sobre una tabla `tasks`. Aquí se explica detalladamente cada parte:

Importación y contexto de la base de datos

```
import { useSQLiteContext } from "expo-sqlite";
```

- Importa el contexto SQLite proporcionado por Expo para acceder la base de datos.
- `useSQLiteContext` es un hook que devuelve el objeto `database` con métodos para ejecutar consultas SQL.

Hook personalizado useDB

```
export function useDB() {  
  const database = useSQLiteContext();  
  // funciones CRUD aquí...  
  return { ... };  
}
```

- `useDB` es una función que crea un contexto para acceder a la base de datos y expone un conjunto de funciones para manipular datos.
- `database` es el objeto que permite hacer consultas SQL asíncronas.

Función `findAll`: obtener todas las tareas

```
async function findAll() {  
  try {  
    const query = "SELECT * FROM tasks";  
    const result = await database.getAllAsync(query);  
    return result;  
  } catch (error) { console.error(error); }  
}
```

- Define una función asíncrona para obtener todas las filas de la tabla `tasks`.

- `getAllAsync(query)`: ejecuta la consulta SQL y retorna todos los registros.
- Se maneja posible error con `try-catch`.

Función `searchByDescription`: buscar tareas por texto

```
async function searchByDescription(text) {  
  try {  
    const query = "SELECT * FROM tasks WHERE description LIKE ?";  
    const result = await database.getAllAsync(query, `${text}%`);  
    return result;  
  } catch (error) { console.error(error); }  
}
```

- Busca filas donde la columna `description` contiene el texto dado.
- Usa parámetro SQL con comodines `%` para búsqueda "contiene".
- Se ejecuta la consulta con parámetro seguro para evitar inyección SQL.

Función `createOne`: crear una nueva tarea

```
async function createOne(task) {  
  const statement = await database.prepareAsync(  
    "INSERT INTO tasks (description) VALUES ($description)"  
  );  
  try {  
    const result = await statement.executeAsync({  
      $description: task.description,  
    });  
    return result;  
  } catch (error) { console.error(error); }  
  finally {  
    await statement.finalizeAsync(); // importante para liberar recursos  
  }  
}
```

- Usa sentencias preparadas para insertar de forma segura un nuevo registro.
- `prepareAsync` crea la consulta con un parámetro nombrado `$description`.

- `executeAsync` ejecuta con el valor real.
- Finalmente se libera la sentencia con `finalizeAsync`.

Función `toggleIsCompleted`: actualizar estado `isCompleted`

```
async function toggleIsCompleted(id, newState) {  
  const query = "UPDATE tasks SET isCompleted = ? WHERE id = ?";  
  const result = await database.getFirstAsync(query, !newState, id);  
}
```

- Actualiza la columna `isCompleted` de la tarea identificada por `id`.
- Se usa `?` para parámetros en la consulta SQL, usando el valor negado de `newState`.
- `getFirstAsync` ejecuta la consulta y retorna el primer resultado (aquí se usa para ejecutar el update).

Función `deleteOne`: eliminar una tarea por id

```
async function deleteOne(id) {  
  try {  
    const result = await database.execAsync(  
      `DELETE FROM tasks WHERE id = ${id}`  
    );  
  } catch (error) { console.error(error); }  
}
```

- Elimina una fila según el `id` recibido.
- Se maneja con `try-catch` para errores.
- Nota: aquí usa interpolación directa, debería usarse parámetro para evitar riesgos de inyección SQL.

Retorno del hook

```
return {  
  findAll,  
  searchByDescription,  
  createOne,  
  toggleIsCompleted,  
  deleteOne  
};
```

- Devuelve un objeto con las funciones que permiten manipular los datos desde componentes React.

Resumen

Este código implementa una capa de acceso a SQLite en React Native con Expo usando promesas y sentencias preparadas. Provee funciones claras para:

- Obtener todas las tareas
- Buscar tareas por texto
- Crear una tarea nueva
- Actualizar el estado completado
- Eliminar una tarea

Así se facilita la interacción con la base de datos de forma segura y asíncrona, manteniendo el código modular y reutilizable para componentes en la app.

```
import { SafeAreaView } from 'react-native-safe-area-context';  
import TodoList from './TodoList';  
  
export default function MainContent() {  
  return (  
    <SafeAreaView style={{flex:1}}>  
      <TodoList />  
    </SafeAreaView>  
  );  
};
```

Análisis detallado del código React Native

Importaciones

- `SafeAreaView` de `react-native-safe-area-context`: este componente crea un contenedor que automáticamente aplica relleno (`padding`) para evitar que el contenido quede tapado por elementos del sistema como la barra de estado, bordes redondeados o notch en dispositivos móviles.
- `TodoList`: componente personalizado, probablemente muestra una lista de tareas o ítems.

Función `MainContent`

- Es un componente funcional que retorna JSX.
- Se retorna un `SafeAreaView` con estilo `{flex: 1}` para que ocupe toda la pantalla disponible.
- Dentro del `SafeAreaView` se renderiza el componente `TodoList`, que será mostrado respetando las áreas seguras del dispositivo.

¿Qué significa `flex:1`?

- Es una propiedad de Flexbox que indica que el componente debe expandirse para ocupar todo el espacio disponible de su padre.

¿Por qué usar `SafeAreaView`?

- En móviles modernos, especialmente en iOS y algunos Android, algunas áreas en pantalla pueden superponerse con elementos del sistema (notch, barras), y hacer que el contenido quede tapado.
- `SafeAreaView` automáticamente agrega espacio para que el contenido esté completamente visible y no chocado con esos elementos.

Resumen

Este código crea un contenedor que respeta las áreas seguras del dispositivo y dentro coloca la lista de tareas, asegurando que la UI sea correcta y visible sin solapamientos incómodos.

```

1 import React, { useState, useEffect } from 'react';
2 import { View, FlatList, StyleSheet } from 'react-native';
3 import Todo from './Todo';
4 import { LinearGradient } from 'expo-linear-gradient';
5 import AppHeader from './AppHeader';
6 import AddTodo from './AddTodo';
7 import { useDB } from '../database/useDB';
8
9 const TodoList = () => {
10   const db = useDB();
11   const [todos, setTodos] = useState([]);
12   const [text, setText] = useState('');
13   const [query, setQuery] = useState('');
14   const addTodo = async () => {
15     if (text.trim()) {
16       await db.createOne({ description: text });
17       setText('');
18       await list();
19     }
20   };
21   const deleteTodo = async (id) => {
22     try {
23       await db.deleteOne(id);
24       list();
25     } catch (error) {
26       console.error(error);
27     }
28   };
29   const toggleIsCompleted = async (id, state) => {
30     try {
31       await db.toggleIsCompleted(id, state);
32       list();
33     } catch (error) {
34       console.error(error);
35     }
36   };
37   const list = async () => {
38     try {
39       const results = await db.findAll();
40       setTodos(results);
41     } catch (error) {
42       console.error(error);
43     }
44   };

```

```

45   const filteredTodos = todos.filter(todo => todo.description.toLowerCase().includes(query.toLowerCase()));
46   useEffect(() => {
47     list();
48   }, []);
49
50   return (
51     <LinearGradient
52       colors={['#dadde4', '#c0d3fd', '#76a9d5', '#512da8']}
53       style={styles.fullWidth}>
54       <AppHeader />
55       <View style={{ flex: 1, padding: 16 }}>
56         <AddTodo text={text} query={query} setText={setText} addTodo={addTodo} setQuery={setQuery} />
57
58         <FlatList
59           data={filteredTodos}
60           keyExtractor={(item) => item.id.toString()}
61           showsVerticalScrollIndicator={false}
62           renderItem={({ item }) => (
63             <Todo {...item} deleteTodo={deleteTodo} toggleIsCompleted={toggleIsCompleted}>{item.description}</Todo>
64           )}
65         />
66       </View>
67     </LinearGradient>
68   );
69
70   const styles = StyleSheet.create({
71     fullWidth: {
72       flex: 1,
73     },
74   });
75
76   export default TodoList;

```

El código de React Native que muestras define un componente funcional llamado `TodoList` que administra una lista de tareas (todos) con operaciones para agregar,

eliminar, marcar como completadas y filtrar basándose en una búsqueda. A continuación te explico detalladamente cada parte:

Importaciones

- `React, { useState, useEffect }`: Importa React y los hooks de estado y efecto para manejar datos y ciclos de vida en componentes funcionales.
- `View, FlatList, StyleSheet`: Componentes básicos de React Native para estructuras de vistas, listas optimizadas y estilos.
- `Todo, AppHeader, AddTodo`: Componentes personalizados importados que corresponden al ítem individual, encabezado de la app y formulario para añadir tareas.
- `LinearGradient` de Expo: Para definir un fondo degradado visual atractivo.
- `useDB`: Un hook personalizado que maneja la persistencia y operaciones sobre la base de datos local de tareas.

Estados internos del componente

```
const [todos, setTodos] = useState([]); // Lista de tareas actuales
const [text, setText] = useState(''); // Texto actual para nueva tarea
const [query, setQuery] = useState(''); // Texto para filtrar tareas
```

Estos estados controlan la lógica interna y la presentación dinámica de la lista.

Funciones clave

- **addTodo**: Inserta una nueva tarea si el texto no está vacío (con trim para evitar espacios solos), limpia el texto en input y recarga la lista desde la DB.
- **deleteTodo**: Recibe el id de una tarea y la elimina de la DB; luego actualiza la lista.
- **toggleIsCompleted**: Cambia el estado de "completado" de una tarea en la DB por id y estado recibido, luego refresca la lista.

- **list:** Obtiene todas las tareas guardadas en la base de datos y actualiza el estado `todos`. Usada al iniciar y cada vez que cambia la data.

Estas funciones usan `async/await` para operaciones asíncronas sobre la base de datos y controlan errores con `try/catch`.

Filtrado de tareas

```
const filteredTodos = todos.filter(todo =>
  todo.description.toLowerCase().includes(query.toLowerCase()));
```

Filtra la lista `todos` según el texto en `query`, ignorando mayúsculas o minúsculas. Esto permite buscar dentro de las descripciones.

Ciclo de vida con `useEffect`

```
useEffect(() => {
  list();
}, []);
```

Se ejecuta una vez al montar el componente, cargando la lista inicial de tareas desde la base de datos.

Renderizado JSX

- `<LinearGradient>`: Envuelve toda la UI con un fondo degradado entre varios tonos.
- `<AppHeader>`: Muestra la cabecera de la aplicación.
- `<View>`: Contenedor con padding para el contenido principal.
- `<AddTodo>`: Componente para ingresar una nueva tarea y barra de búsqueda, recibe props para controlar texto, consulta, y función para agregar tarea.
- `<FlatList>`: Lista optimizada que muestra sólo los ítems visibles; usa `filteredTodos` para renderizar.

- `keyExtractor`: Clave única para cada ítem usando su `id`.
- `renderItem`: Renderiza cada tarea usando el componente `Todo`, pasando funciones para borrar y cambiar estado, además del texto.

Estilos

Solo declara un estilo llamado `fullWidth` que hace que el contenedor degrade ocupe todo el espacio flexible disponible con `flex: 1`.

Exportación

Finalmente exporta el componente `TodoList` para usarlo en otras partes de la aplicación.

Esta estructura es un ejemplo típico de CRUD (Create, Read, Update, Delete) con una lista filtrable, que utiliza React Native para la interfaz y un hook personalizado para manejar el almacenamiento persistente. El uso de hooks y componentes modulares facilita la legibilidad y mantenibilidad del código.

```

1  import { StyleSheet, View } from 'react-native'
2  import { Text } from 'react-native-paper'
3  import { Svg, Path } from 'react-native-svg'
4  import { useSafeAreaFrame } from 'react-native-safe-area-context'
5  import React from 'react'
6
7  export default function AppHeader() {
8    const frame = useSafeAreaFrame()
9    return (
10     <View>
11       <Svg width={frame.width} height={'120'} viewBox={`0 0 ${frame.width} 100`} preserveAspectRatio="none" fill={'#512da8'}>
12         <Path d="M0,0V46.29c47.79,22.2,103.59,32.17,158,28,70.36-5.37,136.33-33.31,206.8-37.5C438.64,32.43,512.34,53.67,583,72.05c69.27,18,138.3,24.88,209.4,13.08,36.15-6,69.85-17.84,104.45-29.34C989.49,25,1113-14.29,1200,52.47V0Z" opacity={80} />
13       </Svg>
14       <Text variant='displaySmall' style={styles.headerTitle}>tareApps</Text>
15     </View>
16   )
17 }
18
19 const styles = StyleSheet.create({
20   headerTitle: {
21     color: "#f2e9e9",
22     position: 'absolute',
23     left: 20
24   }
25 })

```

Vamos a desglosar este código React Native línea por línea para que quede claro qué hace cada parte.

Importaciones

- `import { StyleSheet, View } from 'react-native'`
 - Importa dos componentes básicos: `StyleSheet` para definir estilos, y `View`, que es un contenedor visual básico.
- `import { Text } from 'react-native-paper'`
 - Importa el componente `Text`, pero de la librería `react-native-paper`, que añade estilos y temas mejorados para textos.
- `import { Svg, Path } from 'react-native-svg'`
 - Importa para dibujar gráficos vectoriales: `Svg` es el lienzo, y `Path` define formas vectoriales.
- `import { useSafeAreaFrame } from 'react-native-safe-area-context'`
 - Hook para obtener las dimensiones del área visible segura (sin solapamiento de barras o notch).
- `import React from 'react'`
 - Importa la biblioteca React, necesaria para componentes React.

Definición del componente `AppHeader`

- `export default function AppHeader() {`
 - Declara el componente funcional exportado como predeterminado.
- `const frame = useSafeAreaFrame()`
 - Usa el hook para obtener el ancho y alto del área segura de la pantalla, guardándola en `frame`.
- `return ( ... )`
 - Define qué devuelve el componente para renderizar la UI.

Dentro del `return`

- `<View>`
 - Contenedor principal del header.
- `<Svg width={frame.width} height={'120'} viewBox={0 0 ${frame.width} 100} preserveAspectRatio="none" fill={'#512da8'}>`
 - Renderiza un SVG con:
 - Ancho igual al ancho seguro de la pantalla (`frame.width`).
 - Alto fijo de 120 unidades.
 - `viewBox` define el sistema de coordenadas internas, que se ajusta al ancho.
 - El contenido no mantiene proporción para estirarse horizontalmente (`preserveAspectRatio="none"`).
 - Color de relleno púrpura (`#512da8`).
- `<Path d=... opacity={80} />`
 - Define un camino (forma) SVG con un trazado complejo para crear una curva decorativa en el header.
 - `opacity={80}` indica transparencia, aunque en React Native deberíamos usar valores de 0 a 1; opacidad 80 es inválido y se interpretaría como 80 (muy opaco). Lo correcto sería 0.8.
- `</Svg>`
 - Cierra el SVG.

```
- `<Text variant='displaySmall' style={styles.headerTitle}>tareApps</Text>`
```

- Muestra un texto con el estilo variante "displaySmall" de React Native Paper.
- Aplica estilos personalizados definidos en `styles.headerTitle`.
- El texto dice "tareApps".
- `</View>`

- Cierra el contenedor.

Definición de estilos

- `const styles = StyleSheet.create({ ... })`
 - Define un objeto de estilos.
- `headerTitle: { ... }`
 - Color del texto: casi blanco (`#f2e9e9`).
 - Posición absoluta para superponer al SVG.
 - Alineado a la izquierda con 20 unidades de separación.

```
1 import { Keyboard, StyleSheet, View } from 'react-native';
2 import { TextInput, Button } from 'react-native-paper';
3
4 const AddTodo = ({ text, query, setText, addTodo, setQuery }) => {
5   function viewAll() {
6     setQuery('');
7     setText('');
8     Keyboard.dismiss();
9   }
10  function handleSearch() {
11    setQuery(text);
12    Keyboard.dismiss();
13  }
14  return (
15    <View style={styles.container}>
16      <TextInput
17        multiline
18        mode='flat'
19        activeOutlineColor='tomato'
20        textColor='black'
21        value={text}
22        onChangeText={setText}
23        label="Agregar o Buscar tareas"
24        right={<TextInput.Icon icon="magnify" onPress={handleSearch} />}
25      />
26      {!query ?
27        (<Button mode='contained' collapsable onPress={addTodo} >Agregar tarea</Button>)
28        :
29        (<Button mode='contained' onPress={viewAll} >Ver Todo</Button>)}
30      </View>
31    );
32  };
33
34 export default AddTodo;
35
36 const styles = StyleSheet.create({
37   container: {
38     rowGap: 10,
39     justifyContent: "space-between",
40     marginBottom: 10
41   }
42 });
43
```

Vamos a analizar el código de React Native línea a línea para entender bien qué hace cada parte.

Primero, como introducción breve, este componente llamado `AddTodo` permite agregar o buscar tareas con un campo de texto y dos botones que cambian según si estás buscando (query) o no. Ahora, paso a paso:

1. `import { TextInput, Button } from 'react-native-paper';`
 - Importa dos componentes visuales importantes desde la librería `react-native-paper` que son `TextInput` (campo de texto enriquecido) y `Button` (botón estilizado). Esta librería ofrece componentes con diseño Material Design.
2. `const AddTodo = ({ text, query, setText, addTodo, setQuery }) => {`
 - Define un componente funcional llamado `AddTodo` que recibe como props cinco cosas:
 - `text`: el texto actualmente escrito
 - `query`: cadena para buscar tareas
 - `setText`: función para actualizar `text`
 - `addTodo`: función para agregar una tarea
 - `setQuery`: función para actualizar `query`
3. `function viewAll() {...}`
 - Declara una función que limpia las búsquedas (pone `query` y `text` como vacíos) y oculta el teclado.
4. `function handleSearch() {...}`
 - Asigna al filtro `query` el texto actual (`text`) y oculta el teclado.
5. `return ( ... )`
 - El bloque JSX que describe lo que se va a mostrar en pantalla:
 - `<View style={styles.container}>`: contenedor con estilos.
 - `<TextInput ... />`: área para escribir texto:
 - `multiline`: permite varias líneas
 - `mode='flat'`: estilo plano del input
 - `activeOutlineColor='tomato'`: color del borde al enfocar
 - `textColor='black'`: color del texto
 - `value={text}`: muestra el texto almacenado

- `onChangeText={setText}`: al cambiar el texto, usa la función para actualizar
- `label="Agregar o Buscar tareas"`: etiqueta del input
- `right={<TextInput.Icon icon="magnify" onPress={handleSearch} />}`: un ícono de lupa que al presionarlo ejecuta la búsqueda.

6. Condicional para mostrar botones:

- Si no hay `query` (no estamos buscando), aparece el botón "Agregar tarea", que al presionarse ejecuta `addTodo`.
- Si hay `query`, cambia a botón "Ver Todo" que al presionar limpia todo con `viewAll`.

7. `export default AddTodo`;

- Exporta el componente para que pueda usarse en otras partes de la app.

8. Definición de estilos `styles` con `StyleSheet.create`

- Estilo para el contenedor:
 - `rowGap: 10`: espacio entre elementos en columna (funciona como margen vertical)
 - `justifyContent: "space-between"`: distribuye espacio entre los elementos horizontalmente
 - `marginBottom: 10`: margen abajo para separar visualmente

```
import { Text } from 'react-native-paper';
import MaterialCommunityIcons from '@expo/vector-icons/MaterialCommunityIcons';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
const SWIPE_THRESHOLD = 100;
const Todo = ({ id, description, isCompleted, createdAt, deleteTodo, toggleIsCompleted }) => {
  const showDetails = () => { };
  const handleDelete = (id) => {
    Alert.alert(
      'Eliminar tarea',
      ¿Estás seguro de que deseas eliminar ${id}?,
      [
        {
          text: 'Cancelar',
          style: 'cancel',
        },
      ],
    );
  };
}
```

```

        {
            text: 'Eliminar',
            style: 'destructive',
            onPress: () ⇒ deleteTodo(id)
        }
    ]
);
};

```

//lógica de acciones por desplazamiento

```
const translateX = new Animated.Value(0);
```

```
const panGesture = Gesture.Pan()
```

```

    .onUpdate((event) ⇒ {
        translateX.setValue(event.translationX);
    })
    .onEnd((event) ⇒ {
        const { translationX } = event;
        if (translationX < -SWIPE_THRESHOLD * 2) {
            deleteTodo(id);
        } else if (translationX > SWIPE_THRESHOLD * 2) {
            toggleIsCompleted(id, isCompleted);
        }
        Animated.spring(translateX, {
            toValue: 0, // Regresa a la posición original.
            bounciness: 10, // Grado de rebote al regresar.
            useNativeDriver: false
        }).start();
    });

```

// redLayerOpacity se utiliza para controlar la visibilidad de la capa roja que indica eliminación.

// A medida que el usuario realiza el swipe hacia la izquierda, la capa se vuelve más visible.

```

const redLayerOpacity = translateX.interpolate({
    inputRange: [-SWIPE_THRESHOLD * 2, 0], // Rango de movimiento para la interpolación.
    outputRange: [1, 0], // La opacidad cambia de 1 (completamente visible) a 0 (invisible).
    extrapolate: 'clamp', // Limita los valores interpolados al rango definido.
});

```

// Similar a redLayerOpacity, deleteIconOpacity controla la visibilidad del ícono de eliminación.

```

const deleteIconOpacity = translateX.interpolate({
    inputRange: [-SWIPE_THRESHOLD * 2, 0],
    outputRange: [1, 0],
    extrapolate: 'clamp',
});

```

// purpleLayerOpacity se utiliza para la capa púrpura, que indica que el todo será completado si el swipe es suficientemente largo hacia la derecha.

```

const purpleLayerOpacity = translateX.interpolate({
    inputRange: [0, SWIPE_THRESHOLD * 2],
    outputRange: [0, 1],

```

```

        extrapolate: 'clamp',
    });

// checkIconOpacity controla la visibilidad del ícono que marca el todo como completado.
const checkIconOpacity = translateX.interpolate({
    inputRange: [0, SWIPE_THRESHOLD * 2],
    outputRange: [0, 1],
    extrapolate: 'clamp',
});

// animatedStyle se utiliza para aplicar transformaciones animadas al todo.
// Aquí se aplica una transformación de traslación horizontal basada en translateX.
const animatedStyle = {
    transform: [{ translateX }]
};

return (
    <View style={styles.container}>
        <Animated.View style={[styles.redLayer, { height: 40, opacity: redLayerOpacity }]}>
            <Animated.View style={[styles.deleteIconContainer, { opacity: deleteIconOpacity }]}>

                </Animated.View>
            </Animated.View>

        <Animated.View style={[styles.purpleLayer, { opacity: purpleLayerOpacity, height: 40 }]}>
            <Animated.View style={[styles.checkIconContainer, { opacity: checkIconOpacity }]}>

                </Animated.View>
            </Animated.View>

        <GestureDetector gesture={panGesture}>
            <Animated.View style={[styles.todoContainer, animatedStyle]}>
                <TouchableOpacity
                    onPress={showDetails}
                    onLongPress={() => toggleIsCompleted(id, isCompleted)}
                >

        < Text style={[isCompleted && styles.completed, { width: 300 }] } variant='bodyLarge'>
            {description.length > 35 ? description.slice(0, 35) + '...' : description}
        </Text>
    </TouchableOpacity>

        <MaterialCommunityIcons name="trash-can-outline" size={24} color="black"
            onPress={() => handleDelete(id)} />

        </Animated.View>
    </GestureDetector>
</View >

```

```

    );
  };

export default Todo;

const styles = StyleSheet.create({
  container: {
    position: 'relative', // Posición relativa para permitir que las capas de color se superpongan correctamente
    // al contenedor principal del todo.
  },
  todoContainer: {
    flexDirection: 'row', // Alinea el texto y los iconos en una fila horizontal.
    justifyContent: 'space-between', // Distribuye el espacio entre el texto y el ícono de eliminación.
    alignItems: 'center', // Alinea verticalmente el contenido en el centro.
    paddingVertical: 8, // Espaciado vertical interno del todo.
    marginTop: 20, // Margen superior entre cada todo.
    backgroundColor: "rgb(237, 221, 245)", // Color de fondo del todo.
    borderRadius: 12, // Bordes redondeados para un diseño más amigable.
    paddingLeft: 20, // Espaciado a la izquierda del texto.
    paddingRight: 20, // Espaciado a la derecha del texto.
  },
  completed: {
    textDecorationLine: 'line-through', // Aplica una línea a través del texto para indicar que el todo está
    // completado.
    color: '#512da8', // Cambia el color del texto para indicar su estado de completado.
  },
  redLayer: {
    position: 'absolute', // Posición absoluta para que la capa roja cubra toda el área del todo cuando se realiza
    // el swipe.
    top: 20, // Alineación superior basada en el margen superior del todo.
    right: 0, // La capa roja se alinea al borde derecho del todo.
    left: 0, // La capa roja se extiende hasta el borde izquierdo del todo.
    backgroundColor: 'red', // Color de fondo rojo para indicar una acción destructiva (eliminar).
    borderRadius: 12, // Bordes redondeados para que coincidan con los del todo.
  },
  deleteIconContainer: {
    position: 'absolute', // Posiciona el ícono de eliminación dentro de la capa roja.
    right: 15, // Desplazado ligeramente hacia la izquierda desde el borde derecho.
    top: '20%', // Ajustado verticalmente en la capa.
  },
  purpleLayer: {
    position: 'absolute', // Posición absoluta similar a la capa roja.
    top: 20,
    right: 0,
    left: 0,
    backgroundColor: 'blueviolet', // Color de fondo púrpura para indicar una acción constructiva (completar).
    borderRadius: 12,
  },
  checkIconContainer: {

```

```

    position: 'absolute', // Posiciona el ícono de check dentro de la capa púrpura.
    left: 15, // Desplazado ligeramente hacia la derecha desde el borde izquierdo.
    top: '20%', // Ajustado verticalmente en la capa.
  }

});

```

Este código es un componente funcional React Native llamado `Todo` que representa una tarea con funcionalidad para completar o eliminar mediante gestos de deslizamiento (swipe) y botones. A continuación te explico cada línea y bloque para que entiendas su significado y función.

Importaciones y constantes iniciales

```
import { StyleSheet, View, TouchableOpacity, Alert, Animated } from
'react-native';
```

- Importa componentes básicos y utilidades de React Native:
 - `StyleSheet` para estilos,
 - `View` para contenedores,
 - `TouchableOpacity` para botones táctiles con efecto de opacidad,
 - `Alert` para mostrar cuadros de alerta,
 - `Animated` para animaciones.

```
import { Text } from 'react-native-paper';
```

- Importa el componente `Text` de la biblioteca look & feel `react-native-paper`, que añade variantes estilísticas listas para usar.

```
import MaterialCommunityIcons from
'@expo/vector-icons/MaterialCommunityIcons';
```

- Importa íconos del conjunto Material Community Icons para mostrar íconos gráficos.

```
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
```

- Importa la API avanzada para manejar gestos táctiles en React Native.

```
const SWIPE_THRESHOLD = 100;
```

- Define la distancia mínima de deslizamiento (en píxeles) para que la acción por swipe se considere válida.

Definición del componente Todo y funciones internas

```
const Todo = ({ id, description, isCompleted, createdAt, deleteTodo, toggleIsCompleted }) => {
```

- Componente funcional que recibe props:
 - `id`: identificador de tarea,
 - `description`: texto de la tarea,
 - `isCompleted`: booleano si está completada,
 - `createdAt`: fecha de creación,
 - `deleteTodo`: función para eliminar,
 - `toggleIsCompleted`: función para alternar completado.

```
const showDetails = () => { };
```

- Función vacía preparada para mostrar detalles (puede implementarse después).

```
const handleDelete = (id) => {  
  Alert.alert(  
    'Eliminar tarea',  
    `¿Estás seguro de que deseas eliminar ${id}?`,  
    [  
      { text: 'Cancelar', style: 'cancel' },  
      { text: 'Eliminar', style: 'destructive', onPress: () =>  
deleteTodo(id) }  
    ]  
  );  
};
```

- Función que muestra una alerta confirmando si se desea eliminar la tarea.
- Tiene dos opciones: Cancelar o Eliminar (que ejecuta `deleteTodo`).

Animaciones y gestos para Swipe

```
const translateX = new Animated.Value(0);
```

- Valor animado para el desplazamiento horizontal del componente (inicia en 0).

```
const panGesture = Gesture.Pan()
  .onUpdate((event) => {
    translateX.setValue(event.translationX);
  })
  .onEnd((event) => {
    const { translationX } = event;
    if (translationX < -SWIPE_THRESHOLD * 2) {
      deleteTodo(id);
    } else if (translationX > SWIPE_THRESHOLD * 2) {
      toggleIsCompleted(id, isCompleted);
    }
    Animated.spring(translateX, {
      toValue: 0,
      bounciness: 10,
      useNativeDriver: false
    }).start();
  });
```

- Define el gesto de deslizamiento horizontal (`Pan`).
- Mientras se desliza, actualiza `translateX` con la distancia desplazada.
- Al terminar:
 - Si la distancia fue mayor a dos veces el umbral (hacia izquierda), elimina la tarea.
 - Si fue mayor a dos veces el umbral hacia la derecha, alterna completado.
 - Luego anima el componente para que regrese suavemente a la posición original (0).

Interpolaciones para opacidad en capas e íconos

```
const redLayerOpacity = translateX.interpolate({
  inputRange: [-SWIPE_THRESHOLD * 2, 0],
  outputRange: [1, 0],
  extrapolate: 'clamp',
});
```

- Controla la opacidad de la capa roja que aparece al deslizar hacia la izquierda.
- Se hace visible a medida que `translateX` va de 0 a negativo.

```
const deleteIconOpacity = translateX.interpolate({
  inputRange: [-SWIPE_THRESHOLD * 2, 0],
  outputRange: [1, 0],
  extrapolate: 'clamp',
});
```

- Opacidad del icono de basurero en la capa roja (igual que la capa roja).

```
const purpleLayerOpacity = translateX.interpolate({
  inputRange: [0, SWIPE_THRESHOLD * 2],
  outputRange: [0, 1],
  extrapolate: 'clamp',
});
```

- Opacidad de la capa púrpura que aparece al hacer swipe hacia la derecha (acción de completar).

```
const checkIconOpacity = translateX.interpolate({
  inputRange: [0, SWIPE_THRESHOLD * 2],
  outputRange: [0, 1],
  extrapolate: 'clamp',
});
```

- Opacidad del icono de check dentro de la capa púrpura, igual que la capa púrpura.

Estilo animado para mover el todo horizontalmente

```
const animatedStyle = {
  transform: [{ translateX }]
};
```

- Objeto de estilo que aplica la transformación horizontal animada según `translateX`.

Renderizado del componente

```
return (
  <View style={styles.container}>
```

- Contenedor principal con posición relativa para superponer capas animadas.

```
    <Animated.View style={[styles.redLayer, { height: 40, opacity:
redLayerOpacity }]}>
      <Animated.View style={[styles.deleteIconContainer, { opacity:
deleteIconOpacity }]}>
        <MaterialCommunityIcons name="trash-can-outline" size={24}
color="whitesmoke" />
      </Animated.View>
    </Animated.View>
```

- Capa roja (eliminar) con altura fija y opacidad animada.
- Ícono de basura con opacidad animada dentro de dicha capa.

```
    <Animated.View style={[styles.purpleLayer, { opacity:
purpleLayerOpacity, height: 40 }]}>
      <Animated.View style={[styles.checkIconContainer, { opacity:
checkIconOpacity }]}>
        <MaterialCommunityIcons name="check-circle-outline" size={24}
color="whitesmoke" />
      </Animated.View>
    </Animated.View>
```

- Capa púrpura (completar) y su ícono de check, ambas con animación de opacidad.

```

    <GestureDetector gesture={panGesture}>
      <Animated.View style={[styles.todoContainer, animatedStyle]}>

```

- Componente que detecta el gesto `panGesture`.
- Vista animada que contiene el todo y se mueve horizontalmente.

```

      <TouchableOpacity
        onPress={showDetails}
        onLongPress={() => toggleIsCompleted(id, isCompleted)}
      >
        <Text style={[isCompleted && styles.completed, { width:
300 }]} variant='bodyLarge'>
          {description.length > 35 ? description.slice(0, 35) +
'...' : description}
        </Text>
      </TouchableOpacity>

```

- Texto de la tarea:
 - Al hacer toque simple llama a `showDetails` (vacío).
 - Al toque prolongado alterna completado con `toggleIsCompleted`.
 - Si la descripción es muy larga, la corta y añade "...".
 - Aplica estilo con línea tachada y color si está completado.

```

      <TouchableOpacity>
        <MaterialCommunityIcons name="trash-can-outline" size={24}
color="black"
        onPress={() => handleDelete(id)} />
      </TouchableOpacity>

```

- Botón ícono para eliminar la tarea con alerta de confirmación.

```

    </Animated.View>
  </GestureDetector>
</View >
);

```

- Cierre de los contenedores y detector de gesto.

Exportación y estilos

```
export default Todo;
```

- Exporta el componente para poder usarlo en otras partes de la app.

Estilos principales

```
const styles = StyleSheet.create({
  container: {
    position: 'relative',
  },
  todoContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    paddingVertical: 8,
    marginTop: 20,
    backgroundColor: "rgb(237, 221, 245)",
    borderRadius: 12,
    paddingLeft: 20,
    paddingRight: 20,
  },
  completed: {
    textDecorationLine: 'line-through',
    color: '#512da8',
  },
  redLayer: {
    position: 'absolute',
    top: 20,
    right: 0,
    left: 0,
    backgroundColor: 'red',
    borderRadius: 12,
  },
  deleteIconContainer: {
    position: 'absolute',
    right: 15,
    top: '20%',
  },
  purpleLayer: {
```

```

        position: 'absolute',
        top: 20,
        right: 0,
        left: 0,
        backgroundColor: 'blueviolet',
        borderRadius: 12,
    },
    checkIconContainer: {
        position: 'absolute',
        left: 15,
        top: '20%',
    }
});

```

- Definen estilos:
 - `container` es relativo para superposiciones.
 - `todoContainer` organiza el contenido en fila, con espaciado y bordes redondeados.
 - `completed` aplica tachado y color diferente.
 - `redLayer` y `purpleLayer` son capas absolutas para colores de swipe.
 - `deleteIconContainer` y `checkIconContainer` posicionan los íconos dentro de esas capas.